APT: Adaptive Pruning and Tuning Pretrained Language Models for Efficient Training and Inference

Bowen Zhao Hannaneh Hajishirzi Qingqing Cao

University of Washington; Allen Institute for Artificial Intelligence; Apple (work done at the University of Washington)

May 24, 2026

LLM quality gains come with prohibitive compute and memory costs

- LLMs power search, coding, assistants, and enterprise analytics.
- Scaling parameters boosts quality but inflates compute and memory.
- Cost, energy, and accessibility drive demand for efficient training and inference.

Practical fine-tuning and serving pipelines hit efficiency walls

- Fine-tuning 7B–13B LMs needs tens of GBs, long runtimes.
- Inference remains costly even with PEFT; base weights dominate.
- Structured pruning helps inference but often slows/complicates training.

We target a unified reduction in training and inference cost

- Goal: jointly reduce training time/memory and inference time/memory.
- PEFT (e.g., LoRA): low training memory but no inference gains.
- Structured pruning: inference speedups but higher training cost and complexity.

Why existing combinations fall short

- Post-hoc prune+distill: slow, memory-heavy, multi-stage pipelines.
- Uniform adapter growth: wastes budget, slow convergence.
- Salience heuristics miss outliers; degrade accuracy under sparsity.

Adaptive pruning with adaptive tuning early in fine-tuning closes the train–serve gap

- Prune unimportant blocks to shrink inference cost while training.
- Add tuning capacity only where it matters to recover accuracy.
- We do both adaptively using low-cost, outlier-aware salience signals.

APT decides what to prune and where to grow during early fine-tuning

- Early-stage adaptive pruning guided by outlier-aware salience.
- Concurrent adaptive tuning: grow ranks only in salient adapters.
- Lightweight self-distillation to recover pruning losses efficiently.
- Takeaway: APT jointly prunes structure and grows capacity as soon as training begins.

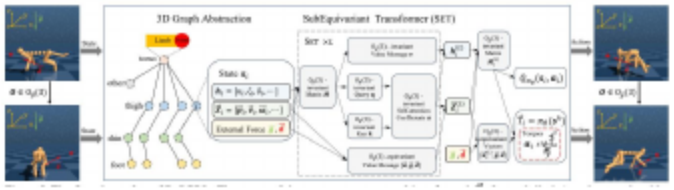

Figure: APT在LM微调中自适应剪枝与调参：显著性二值掩码剪枝+动态秩适配器

APT adapters prune hidden structure and grow rank on demand

- Extend LoRA with binary masks on input/output dims to prune hidden dims, heads, and FFN neurons.
- Dynamic rank growth r_apt adds capacity only where helpful.
- Placement: Place in Q/V and FFN (smaller models).

Outlier-aware salience + budgeted pruning hit target sparsity reliably

- Salience proxy: activation–gradient product on frozen blocks with kurtosis to emphasize critical outliers.
- Compare fairly via per-parameter salience density across blocks.
- Sort blocks by density; prune lowest first; binary search to satisfy strict head/neuron/dim sparsity budgets.
- Smooth, cubic sparsity schedules with mask decay for stability.

Salience-guided rank growth + shared-weight distillation recover accuracy efficiently

- Rank growth: score adapters via parameter–gradient salience; increase ranks only in top-salience half to a tuning budget.
- Safe initialization keeps pre-addition outputs unchanged for stability.
- Shared-weight self-distillation: teacher and student share frozen base; duplicate only adapters with random layer mapping + supervised loss.
- Substantially lowers peak memory versus full-teacher setups while restoring accuracy.

Stability tricks enable smooth structural changes

- Early pruning accelerates training; tuning ranks grow gradually.
- EMA-smoothed salience reduces variance; smooth mask decay avoids shocks.
- Joint constraints on pruned params and tuning budgets over steps.

We evaluate on standard NLP benchmarks with accuracy and efficiency metrics

- Datasets:
  - For training: Alpaca
  - For evaluation: GLUE (SST-2, MNLI), SQuAD v2, CNN/DM, ARC, HellaSwag, MMLU, TruthfulQA.
- Baselines span strong PEFT, pruning, and combined methods (e.g., LoRA, LoRA+Prune, CoFi/LLMPruner, Prune+Distill).
- Metrics: task accuracy, training time/memory, inference speed/memory with hardware-aligned structured pruning.

APT matches dense accuracy while cutting end-to-end cost on RoBERTa/T5 at 60% sparsity

- Takeaway: APT matches near-dense accuracy while cutting end-to-end cost (faster fine-tuning and real inference gains).

Table: RoBERTa and T5 pruning with APT compared to baselines under 60% sparsity.

Model	Method	MNLI	SST2	SQuAD v2	CNN/DM	Train Time↓	Train Mem↓	Inf Time↓	Inf Mem↓
RoBERTabase	FT	87.6	94.8	82.9	-	100.0%	100.0%	100.0%	100.0%
	LoRA	87.5	95.1	83.0	-	2137.0%	60.5%	100.0%	100.0%
	LoRA+Prune	84.0	93.0	79.2	-	5128.3%	60.5%	38.0%	75.1%
	Prune+Distill	87.3	94.5	-	-	1495.3%	168.5%	38.6%	79.2%
	LoRA+Prune+Distill	84.2	91.9	-	-	6534.6%	141.4%	39.4%	82.3%
	APT	86.4	94.5	81.8	-	592.1%	70.1%	41.3%	78.1%
T5base	FT	87.1	95.2	-	42.1/20.3/39.4	100.0%	100.0%	100.0%	100.0%
	LoRA	87.0	95.0	-	38.7/17.2/36.0	255.5%	62.0%	100.0%	100.0%
	LoRA+Prune	80.9	92.3	-	36.7/15.7/33.9	4523.5%	62.0%	47.1%	73.4%
	APT	87.0	95.0	-	38.6/17.0/35.8	484.7%	73.9%	74.6%	81.5%

APT improves average score and reduces memory on LLaMA-2 7B at 30% sparsity

- Takeaway: On 7B models, APT improves average scores over pruning baselines and reduces training/inference memory versus LoRA.

Table: LLaMA-2 7B at 30% sparsity (trained on Alpaca; evaluated on ARC, HellaSwag, MMLU, TruthfulQA).

Method	ARC	HellaSwag	MMLU	TruthfulQA	Avg.	Train Time↓	Train Mem (↓)	Inf Time↓	Inf Mem↓
LLaMA 2 7B	53.1	77.7	43.8	39.0	53.4	-	-	-	-
LoRA	55.6	79.3	46.9	49.9	57.9	100.0%	100.0%	100.0%	100.0%
LoRA+Prune	46.8	65.2	23.9	46.2	45.5	180.9%	100.0%	115.5%	68.9%
LLMPruner	39.2	67.0	24.9	40.6	42.9	86.9%	253.6%	114.8%	74.2%
APT	45.4	71.1	36.9	46.6	50.0	106.0%	75.8%	117.0%	67.2%

Holistic gains: accuracy–efficiency Pareto and end-to-end cost–performance

- Takeaway: APT shifts the Pareto frontier and improves end-to-end throughput/memory at strong accuracy.

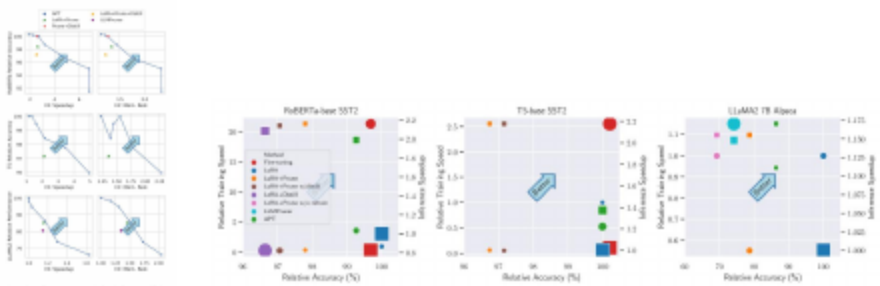

Figure: RoBERTa/T5/LLaMA-2 7B的任务表现与相对推理效率；以及APT相对基线的端到端训练/推理权衡。

Salience-timed pruning and growth beat static choices

- Takeaway: Early pruning and salience-guided rank growth converge faster and reduce memory versus large fixed ranks.

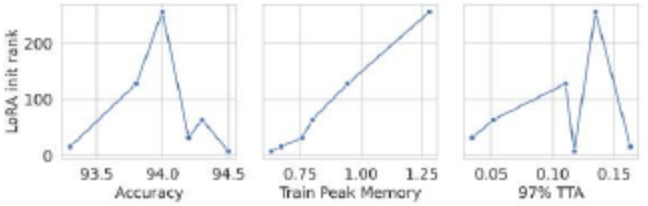

Figure: APT对RoBERTa/SST2中LoRA初始秩剪枝比较：准确率、相对峰值显存与达97%微调精度速度

Outlier-aware salience and adaptive tuning drive the gains

- Outlier-aware salience (kurtosis) is critical to preserve performance under moderate/high sparsity.
- Adaptive Tuning (rank growth) consistently improves accuracy versus fixed ranks at equal sparsity.
- Adaptive Pruning drives inference gains; removing it erases speedups and raises training memory.
- Shared-weight self-distillation recovers accuracy with modest overhead versus full-teacher distillation.

Dynamic changes risk instability; hardware alignment and richer adapters extend gains

- Dynamic structure changes can introduce optimization instability; grouping (e.g., multiples of 8/16) helps wall-clock gains.
- Remaining performance gap at very high sparsity and very large LMs; explore smarter schedules and nonlinear adapters.
- Broaden to more tasks and memory-lean distillation for billion-scale models.

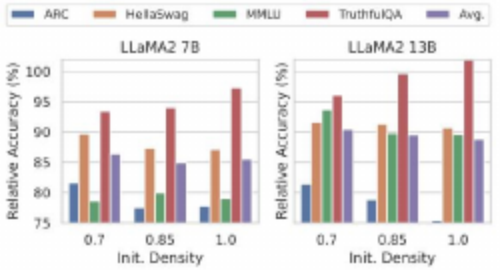

Figure: APT分析：不同初始/目标稀疏度与自适应调度

Adaptive pruning+tuning delivers unified efficiency for LMs

- APT prunes and tunes simultaneously to cut training and inference costs.
- Outlier-aware salience and selective rank growth preserve accuracy.
- Empirically strong across RoBERTa, T5, and LLaMA with real speedups and memory savings.

We welcome questions and discussion

- Thank you for your attention!
- Questions and feedback are welcome.

Community support and open-source release

- We thank collaborators and the open-source community.
- Code and models: `https://github.com/ROIM1998/APT`